

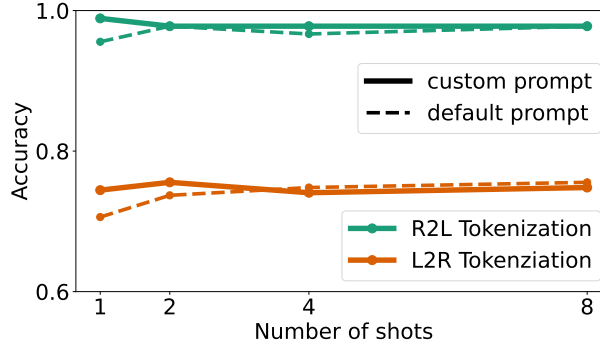


Figure 16. Comparison of R2L and L2R tokenization strategies for different numbers of shots and using two different system prompts.

C. Frequency effects

Given the findings of prior work on numerical reasoning demonstrating frequency effects (Razeghi et al., 2022), we also investigated whether or not our observed error patterns could be explained by term frequency. While we do not have access to the pre-training data of GPT-3.5 and GPT-4 models, we use the tokenizer merge ranks¹⁷ as a signal of term frequency. We analyze the expanded set of problems used for the error analysis in Section 4. Our results are summarized below:

When making an error, GPT-3.5 is slightly more likely to output a more frequent token. For each token in the model response on problems where it makes a mistake, we consider if the outputted incorrect token is more or less frequent (has lower or higher merge rank) than the correct one. Of the 25 errors made by the model when using R2L tokenization, 15 involve substituting in a more frequent token (60%, $p = 0.115$ using a binomial null distribution assuming chance is 50%). Of the 425 errors made when using L2R tokenization, 238 involve substituting in a more frequent token (56%, $p = 0.005$ using a binomial null distribution assuming chance is 50%). While we do see a significant effect in the L2R tokenization case, the margin is relatively small, which suggests that token frequency is not the dominant reason behind the error patterns.

When using L2R tokenization in the length mismatch case, GPT-3.5 errors do not show strong correlation to token frequency. In Section 4.3, we found that GPT-3.5 always gets the fourth digit wrong (Figure 9a). We then found correlation in the specific error in digit 4 to the magnitude difference between correct digit and digit in the model response (Figure 9b). Here, we ask if the substituted token 2 (whose first digit would be digit 4 of the response) is correlated to frequency in training data. Specifically, for each problem, we rank the tokens corresponding to the 10 possible “digit 4 mistakes” by merge rank. In Figure 17, we show the distribution of ranks across all 365 problems where the model makes “digit 4” errors. If models are preferentially substituting in more frequent tokens, we would expect to see a negative trend from the top left to the bottom right (as we did in Figure 9b). In Figure 17, we see a slight preference for outputting the most likely token (roughly 16% of the time, where chance would be 10%), but overall we see no clear trend.

Off-by-one errors do not seem to be correlated to answer token frequency. In Section 4.4, we found that the vast majority of remaining errors (for R2L tokenization, and for L2R tokenization in the length match condition) are off-by-one errors in the units digit of a token. Here, we ask if the specific substitution by the model is correlated to token frequency, measured by merge rank. Specifically, we condition on the model possibly making an off-by-one error, which means there are 3 possible output tokens (the correct token, the correct token minus one, the correct token plus one). We then rank these tokens based on merge rank, and see if the model preferentially picks the token with lowest merge rank. Of the 24 off-by-one errors when using R2L tokenization, we find the model only picks the “most frequent” token 7 times. Of the 53 off-by-one errors when using L2R tokenization, we find the model only picks the “most frequent” token 17 times. Both of these are essentially what we would expect by chance (one out of three), which suggests that output token frequency effects are not a dominant factor in why the model makes off-by-one errors.

Overall, we find mild to no evidence of token frequency effects in our experiments. This could be due to the presumably larger scale of GPT-3.5 (as compared to GPT-J, used by Razeghi et al. (2022)). However, we note that our method of measuring token frequency is imperfect—relying on BPE merge ranks to signal frequency as we do not have access to

¹⁷Recall that tokens are created roughly in order of decreasing frequency in the corpus used to train the tokenizer.

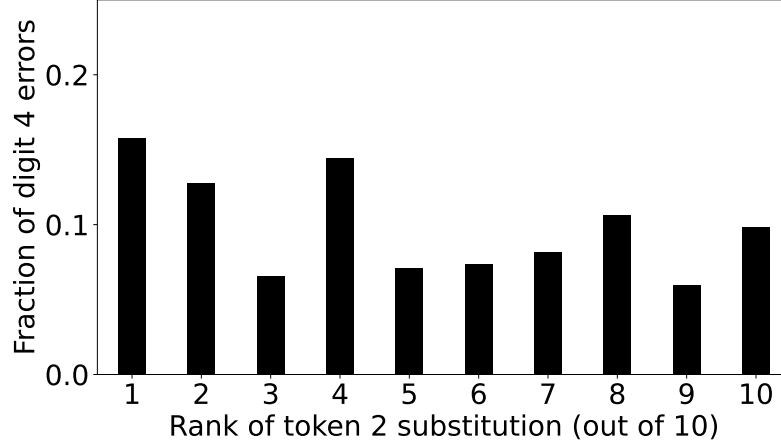


Figure 17. Distribution of relative rank of substituted incorrect token 2 in model response when using L2R tokenization in the length mismatch condition.

pre-training data. Future work could study such associations further in newer, larger models with open pretraining data (Groeneveld et al., 2024).

D. Stereotyped patterns in model log probabilities

Mirroring the results of Section 4, we found stereotyped patterns in model log probabilities (“logprob”). Specifically, the OpenAI API returns the top 5 tokens at each position with their corresponding logprobs. We analyzed these log probabilities in three cases: L2R tokenization on length mismatch problems, L2R tokenization on length match problems, and R2L tokenization on all problems. These conditions mirror the most salient error effects we found in Section 4, with the former leading to “digit 4” errors, and the latter two leading to mostly off-by-one errors.

In addition to the raw logprobs, we computed an additional entropy metric (per output token) to measure model uncertainty in its output. Since access is restricted to the top 5 logprobs, we use the following lower bound, H_{lower} , to the true entropy:

$$\begin{aligned}
 H_{\text{true}} &\equiv - \sum_{i=1}^V p_i \log(p_i) \\
 &= - \left(\sum_{i=1}^5 p_i \log(p_i) + \sum_{i=6}^V p_i \log(p_i) \right) \\
 &\geq - \left(\sum_{i=1}^5 p_i \log(p_i) + \sum_{i=6}^V p_i \log(p_5) \right) \\
 &= - \left(\sum_{i=1}^5 p_i \log(p_i) + \left(1 - \sum_{i=1}^5 p_i \right) \cdot \log(p_5) \right) \\
 &\equiv H_{\text{lower}},
 \end{aligned}$$

where p_i denotes the probability of the i -th most likely token. We use the natural logarithm for entropy, so all entropies are in nats (not bits).

For the “digit 4” error pattern (Section 4.3), we find an interesting trend in model entropy. The entropy both on problems it gets incorrect (91.25%) and correct (8.25%) is roughly the same (2.066 and 2.061 respectively). Even when the model gets the question right, it’s unsure of its answer, suggesting that it might just be guessing a second output token with the right tens and ones digit and random hundreds digit. Providing further evidence for this mechanism, we observe that, of the problems where the model makes an error, about half (49.6%) of the time the correct answer appears in the top 5 output tokens. This is in line with what we would see for random guessing from the 10 tokens. That said, the model may exhibit some degree of bias towards the correct output, as evidenced by the downward trend in Figure 9b.

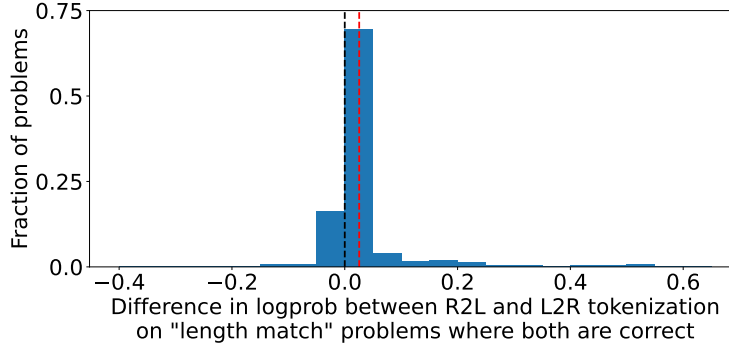


Figure 18. Histogram of difference in answer log probabilities (summed over tokens) on length match problems that the model answers correctly using both R2L and L2R tokenization. Black dotted line signifies 0. Red dotted line shows the average difference—on average, the model is more “confident” when using R2L tokenization.

For the off-by-one error patterns (Section 4.4), we observe a qualitatively different trend. Specifically, of the 53 off-by-one errors when using L2R tokenization on length match problems, in all cases we find that the second most likely token is the correct answer. We observe the same effect on the 25 off-by-one errors when using R2L tokenization. Furthermore, the entropy in both cases is around 0.45 ± 0.05 , indicating that the model puts most of its weight on these top 2 most likely tokens. Unlike in the “digit 4” case, model entropy on correct problems is significantly lower (approximately 0.03, averaged across dataset and tokens) indicating that the model is “confidently correct” when using L2R tokenization on length match problems or R2L tokenization on all problems. Interestingly, on the subset of length match problems that the model answers correctly in both L2R and R2L tokenization, we found the model is slightly more confident when using R2L tokenization (which aligns with our intuition, as the model is also more often correct when using R2L tokenization)—see Figure 18.

These results demonstrate that, depending on tokenization direction and alignment between input and output tokenization, we observe stereotyped patterns in model log probabilities. When using L2R tokenization on length mismatch problems, the model appears to make a magnitude-biased guess between all possible fourth digits (corresponding to 10 possible tokens¹⁸). In the other cases, the model is mostly confidently correct. When it does make an error, it’s almost always an off-by-one error (Section 4.4) where it’s uncertain between its chosen off-by-one incorrect answer and the true answer, but does not really consider other outputs beyond these two.¹⁹

E. Additional experimental details

All code and raw results can be found at <https://github.com/aadityasingh/TokenizationCounts>.

E.1. Length control for error analysis

As described in Section 4.1, after noticing errors mostly come from the length mismatch condition in our original experiments (which used 90 problems, balanced by input digit length), we conducted a larger experiment where we controlled for input and output digit lengths. Specifically, we considered the following (addend1_length, addend2_length, answer_length) triplets: (7,7,7), (7,7,8), (8,7,8), (7,8,8), (8,7,9), (7,8,9), (8,8,8), (8,8,9), (9,7,9), (7,9,9), (9,8,9), (8,9,9), (9,9,9). Problems in each condition were sampled randomly so as to satisfy the digit length constraints for each triplet. We sampled 100 problems for each triplet, for a total of 1300 problems.

E.2. Access dates

Given the changing nature of the OpenAI API, we report access dates for all experiments below. We tried to use the supposed “fixed” models for all experiments, but did notice some non-determinism, even at temperature 0—an issue that may be due to non-determinism in floating point arithmetic. We also note that the gpt-4-0314 appears to have

¹⁸A completely random guess over 10 tokens would correspond to an entropy of about 2.3, which is in line with the lower bound observed (of about 2.06) and the finding of a slight magnitude bias (which would decrease the entropy from 2.3).

¹⁹A completely random guess over 2 tokens would correspond to an entropy of about 0.69, which is in line with the lower bound observed (of about 0.45).